Qué es

Pseudo-código (pseudo = falso) se refiere a *cualquier* representación de un algoritmo.

Existe pseudo-código `no gráfico` (como el que hemos visto) y `gráfico` (diagramas de flujo).

Algoritmos de uso real

Hasta ahora sólo hemos visto algoritmos basados en acciones de la vida real.

Sin embargo, los algoritmos de la programación no son tan distintos como podrías pensar.

La diferencia principal es que siguen una estructura más rígida y se ordenan de distintas maneras.

Por ejemplo, analicemos este código de Javascript (no tienes que entenderlo, solo quiero demostrar que por debajo, no es más que un algoritmo como los que hemos visto):

```
function onButtonClick() {
  const numberLabelElement = document.getElementById("number-la
   const currentNumberText = numberLabelElement.innerHTML;
   const currentNumber = Number(currentNumberText);
   const newNumber = currentNumber + 1;
   numberLabelElement.innerHTML = newNumber;
}
```

Copiar

Podemos observar que Javascript, al ser un lenguaje de programación, tiene cierta `sintaxis` especifica que se debe seguir.

Sin embargo, cada línea es una instrucción, justo como las que hemos visto.

Si quisiéramos pasar este código a pseu-código, se vería algo así:

Hacer que botón sume click

1. Inicio (Cada vez que se haga click en el botón):

2. Buscar el texto que muestra el numero de clicks.
3. Guardar su valor.
4. Convertir su valor a numero.
5. Guardar el valor de la suma del numero + 1.
6. Cambiar al valor del texto a este numero numero.
7. Fin

```
function onButtonClick() {
  const numberLabelElement = document.getElementById("number-la
   const currentNumberText = numberLabelElement.innerHTML;
   const currentNumber = Number(currentNumberText);
   const newNumber = currentNumber + 1;
   numberLabelElement.innerHTML = newNumber;
}
```

Copiar

Especificación de pseudo-código

Con el fin de acercarnos más a código real como este, vamos a ver algoritmos usando una especificación de pseudocódigo no gráfico más estricta.

Vamos a escribir algoritmos utilizando solo las siguientes instrucciones:

Palabra	Uso
`Inicio`	Comienzo del algoritmo
`Fin`	Final del algoritmo
`Definir <nombre>`	Definir una variable
`Mostrar <texto>`	Mostrar algo en pantalla
`Pedir <nombre>`	Pedirle al usuario que ingrese un valor para una variable
`Si (<condición>) entonces: <instrucciones>`	Realizar acciones solo si una condición se cumple, o realizar otra.

Palabra	Uso
`Sino` `<instrucciones alternativas>` `FinSi`	
`Mientras que (<condicion>) repetir:` `<instrucciones>` `FinMientras`	Realizar acciones de manera repetida hasta que una condición se deje de cumplir

Para las condiciones, vamos a usar solo las siguientes comparaciones:

Operación	Símbolo	Descripción	Ejemplos
Igual a	`==`	Revisa si dos valores son iguales	10 == 10 → Verdadero 20 == 20 → Verdadero 10 == 20 → Falso
Distinto a	`!=`	Revisa si dos valores son distintos	10 != 10 → Falso 20 != 20 → Falso 10 != 20 → Verdadero
Mayor a	`>`	Revisa si un numero es mayor a otro	90 > 40 → Verdadero 40 > 90 → Falso 50 > 50 → Falso
Mayor o igual a	`>=`	Revisa si un numero es mayor o igual a otro	90 >= 40 → Verdadero 40 >= 90 → Falso 50 >= 50 → Verdadero
Menor a	`<`	Revisa si un numero es menor a otro	90 < 40 → Falso 40 < 90 → Verdadero 50 < 50 → Falso
Menor o igual a	`<=`	Revisa si un numero es	90 <= 40 → Falso 40 <= 90 → Verdadero

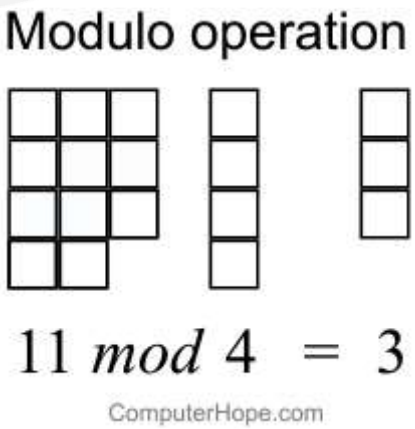
Operación	Símbolo	Descripción	Ejemplos
		menor o igual a otro	50 <= 50 → Verdadero

Tambien podemos hacer operaciones aritméticas:

Operación	Símbolo	Descripción	Ejemplos
Suma	`+`	Suma dos números	10 + 35 → 45
Resta	`-`	Resta un numero de otro	90 - 50 → 40
Multiplicación	`*`	Multiplica dos números	3 * 5 → 15
Division	`/`	Divide un numero por otro	10 / 2 → 5
Potencia	`**`	Eleva un numero a la potencia de otro	2 ** 3 → 8 (2 * 2 * 2)
Mod (Modulo)	`%`	Divide un numero por otro pero retorna su residuo (explicación más a fondo abajo)	10 % 3 → 1
Division entera	`//`	Divide un numero por otro pero quitando	10 // 3 → 3 (3.33)

Operación	Símbolo	Descripción	Ejemplos
		los decimales del resultado	

Modulo



Variables

Nos permiten guardan datos a los que podemos usar en futuras instrucciones del algoritmo.

> Por así decirlo, es como apuntar algo en una nota.
>
> O como meter un objeto en una caja.

Todas las variables tiene un `identificador` (su nombre).

> *Su nombre solo puede contener letras, números o guiones bajos.*

Tambien nos permiten hacer comparaciones y operaciones matemáticas.

Por ejemplo, supongamos que tenemos un algoritmo en el que el que le pedimos su `edad` al usuario, y le mostramos si es mayor de edad o no:

Mostrar si es mayor de edad

1. Inicio
2. Definir edad
3. Mostrar “Cual es su edad?”
4. Pedir edad
5. Si (edad ≥ 18) entonces:

    1. Mostrar “Eres mayor de edad”
6. Sino:
    1. Mostrar “Eres menor de edad”
7. FinSi
8. Fin

En la línea `2` definimos que tendremos una variable llamada `edad.`

El la línea `4`, le pedimos al usuario que la ingrese

En la línea `5`, revisamos si la edad ingresada es mayor o igual a 18

Veamos otro ejemplo:

A un trabajador le descuentan 10% si su sueldo es menor o igual a 1000.
Por encima de 1000 y debajo de 2000, le descuentan 5% del adicional.
Por encima de 2000, el 3% del adicional.
Vamos a hacer un algoritmo que calcule el descuento y el sueldo neto que recibe el trabajador.

Mostrar sueldo neto

1. Inicio
2. Definir `sueldo`
3. Definir `sueldo_neto`
4. Definir `descuento`
5. Mostrar “Ingrese su sueldo”
6. Pedir `sueldo`
7. Si (`sueldo` ≤ 1000) entonces:
    1. `descuento` = sueldo * 0.1
8. Sino:
    1. Si (`sueldo` < 2000) entonces:
        1. `descuento` = (`sueldo` - 1000) * 0.05 + 100
    2. Sino:
        1. `descuento` = 150 + (`sueldo` - 2000) * 0.03

3. FinSi

9. FinSi
10. `sueldo_neto` = `sueldo` - `descuento`
11. Mostrar “El sueldo neto es de”
12. Mostrar `sueldo_neto`
13. Fin